RESTful API Weather Application

Implementing GET, POST, PUT & DELETE requests to a public REST API.

Project Setup

1. **Create project directory structure**
    - Create a new folder for your project
    - Create two files in this folder:
        - `index.html` (for the user interface)
        - `script.js` (for the JavaScript code)

2. **Understanding the APIs we'll use**
    - Weather data: OpenWeatherMap API (for GET requests)
    - Saved locations: JSONPlaceholder API (for POST, PUT, DELETE requests)

Step 1: Building the HTML Structure

The HTML file provides the structure and UI for our application with:

- A tabbed interface for different operations
- Forms for entering data
- Display areas for showing results
- Basic CSS styling

Key HTML components:

- Tab navigation for different API operations
- Weather lookup form (GET request)
- Location saving form (POST request)
- Saved locations display (PUT and DELETE requests)
- Response information display
- Location editing modal

Step 2: Creating the JavaScript Functionality

The JavaScript file implements:

- Event listeners to handle user interactions
- Functions for each API operation (GET, POST, PUT, DELETE)
- Data processing and display logic

index.html

```html
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Weather API Dashboard</title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            max-width: 800px;
            margin: 0 auto;
            padding: 20px;
            background-color: #f5f5f5;
        }

        h1 {
            color: #2c3e50;
            text-align: center;
        }

        .container {
            background-color: white;
            border-radius: 8px;
            box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
            padding: 20px;
            margin-bottom: 20px;
        }

        .form-group {
            margin-bottom: 15px;
        }

        label {
            display: block;
            margin-bottom: 5px;
            font-weight: bold;
        }

        input,
        select,
        button {
            width: 100%;
            padding: 8px;
            border: 1px solid #ddd;
            border-radius: 4px;
            box-sizing: border-box;
        }
```

```
49
50          button {
51              background-color: #3498db;
52              color: white;
53              border: none;
54              cursor: pointer;
55              margin-top: 10px;
56              padding: 10px;
57              font-weight: bold;
58          }
59
60          button:hover {
61              background-color: #2980b9;
62          }
63
64          .weather-display {
65              margin-top: 20px;
66          }
67
68          .weather-card {
69              background-color: #f9f9f9;
70              border-left: 4px solid #3498db;
71              padding: 15px;
72              margin-bottom: 10px;
73              border-radius: 4px;
74          }
75
76          .saved-locations {
77              display: flex;
78              flex-wrap: wrap;
79              gap: 10px;
80              margin-top: 15px;
81          }
82
83          .location-item {
84              background-color: #eef7ff;
85              border: 1px solid #bde0fe;
86              border-radius: 4px;
87              padding: 10px;
88              width: calc(50% - 10px);
89              box-sizing: border-box;
90              position: relative;
91          }
92
93          .location-actions {
94              display: flex;
95              justify-content: space-between;
96              margin-top: 10px;
97          }
```

```
98
99          .btn-edit,
100         .btn-delete {
101             width: 48%;
102             padding: 5px;
103         }
104
105         .btn-delete {
106             background-color: #e74c3c;
107         }
108
109         .btn-delete:hover {
110             background-color: #c0392b;
111         }
112
113         .response-info {
114             background-color: #eefaf7;
115             padding: 10px;
116             border-radius: 4px;
117             margin-top: 15px;
118             font-family: monospace;
119             white-space: pre-wrap;
120         }
121
122         .tabs {
123             display: flex;
124             margin-bottom: 15px;
125         }
126
127         .tab {
128             padding: 10px 15px;
129             background-color: #ddd;
130             cursor: pointer;
131             border-radius: 4px 4px 0 0;
132             margin-right: 5px;
133         }
134
135         .tab.active {
136             background-color: #3498db;
137             color: white;
138         }
139
140         .tab-content {
141             display: none;
142         }
143
144         .tab-content.active {
145             display: block;
146         }
147     </style>
148 </head>
```

```html

<body>
    <h1>Weather API Dashboard</h1>

    <div class="container">
        <div class="tabs">
            <div class="tab active" data-tab="get">GET Weather</div>
            <div class="tab" data-tab="post">POST Location</div>
            <div class="tab" data-tab="saved">Saved Locations</div>
            <div class="tab" data-tab="api-info">API Info</div>
        </div>

        <!-- GET Weather Tab -->
        <div class="tab-content active" id="get-tab">
            <h2>Get Current Weather (GET Request)</h2>
            <div class="form-group">
                <label for="city-input">City Name:</label>
                <input type="text" id="city-input" placeholder="Enter city name (e.g., London)">
            </div>
            <button id="get-weather">Get Weather</button>

            <div class="weather-display" id="weather-result"></div>
        </div>

        <!-- POST Location Tab -->
        <div class="tab-content" id="post-tab">
            <h2>Save Location (POST Request)</h2>
            <div class="form-group">
                <label for="location-name">Location Name:</label>
                <input type="text" id="location-name" placeholder="Enter location name">
            </div>
            <div class="form-group">
                <label for="location-city">City:</label>
                <input type="text" id="location-city" placeholder="Enter city">
            </div>
            <div class="form-group">
                <label for="location-country">Country:</label>
                <input type="text" id="location-country" placeholder="Enter country code (e.g., US)">
            </div>
            <div class="form-group">
                <label for="location-notes">Notes:</label>
                <input type="text" id="location-notes" placeholder="Add optional notes">
            </div>
            <button id="save-location">Save Location</button>
        </div>
```

```html

        <!-- Saved Locations Tab -->
        <div class="tab-content" id="saved-tab">
            <h2>Saved Locations</h2>
            <p>Manage your saved locations (PUT and DELETE requests)</p>
            <div id="saved-locations" class="saved-locations">
                <!-- Saved locations will be displayed here -->
            </div>
        </div>

        <!-- API Info Tab -->
        <div class="tab-content" id="api-info-tab">
            <h2>API Information</h2>
            <p>This application demonstrates RESTful API operations:</p>
            <ul>
                <li><strong>GET</strong> - Retrieves weather data from OpenWeatherMap API</li>
                <li><strong>POST</strong> - Saves a new location to JSONPlaceholder API</li>
                <li><strong>PUT</strong> - Updates an existing saved location</li>
                <li><strong>DELETE</strong> - Removes a saved location</li>
            </ul>
            <p>Last request information:</p>
            <div id="response-info" class="response-info">No requests made yet.</div>
        </div>
    </div>

    <!-- Edit Location Modal (simplified inline) -->
    <div id="edit-modal"
        style="display: none; position: fixed; top: 0; left: 0; width: 100%; height: 100%; background-color: rgba(0,0,0,0.5); z-index: 100;">
        <div
            style="background-color: white; margin: 10% auto; padding: 20px; width: 80%; max-width: 500px; border-radius: 8px;">
            <h3>Edit Location</h3>
            <div class="form-group">
                <label for="edit-name">Location Name:</label>
                <input type="text" id="edit-name">
            </div>
            <div class="form-group">
                <label for="edit-city">City:</label>
                <input type="text" id="edit-city">
            </div>
            <div class="form-group">
                <label for="edit-country">Country:</label>
                <input type="text" id="edit-country">
            </div>
            <div class="form-group">
                <label for="edit-notes">Notes:</label>
                <input type="text" id="edit-notes">
            </div>
            <input type="hidden" id="edit-id">
            <div style="display: flex; justify-content: space-between;">
                <button id="update-location" style="width: 48%;">Update</button>
                <button id="cancel-edit" style="width: 48%; background-color: #95a5a6;">Cancel</button>
            </div>
        </div>
    </div>

    <script src="script.js"></script>
</body>

</html>
```

script.js

```javascript
// Configuration and Constants
const WEATHER_API_KEY = 'API Key'; // Replace with your API key
const WEATHER_API_URL = 'https://api.openweathermap.org/data/2.5/weather';
const PLACEHOLDER_API_URL = 'https://jsonplaceholder.typicode.com/posts';

// Global state to store saved locations
let savedLocations = [];

// DOM Elements
document.addEventListener('DOMContentLoaded', () => {
    // Tab navigation
    const tabs = document.querySelectorAll('.tab');
    tabs.forEach(tab => {
        tab.addEventListener('click', () => {
            const tabId = tab.getAttribute('data-tab');

            // Update active tab
            tabs.forEach(t => t.classList.remove('active'));
            tab.classList.add('active');

            // Update active content
            document.querySelectorAll('.tab-content').forEach(content => {
                content.classList.remove('active');
            });
            document.getElementById(`${tabId}-tab`).classList.add('active');
        });
    });

    // GET Request - Weather data
    document.getElementById('get-weather').addEventListener('click', getWeather);

    // POST Request - Save location
    document.getElementById('save-location').addEventListener('click', saveLocation);

    // Edit Modal Event Listeners
    document.getElementById('update-location').addEventListener('click', updateLocation);
    document.getElementById('cancel-edit').addEventListener('click', () => {
        document.getElementById('edit-modal').style.display = 'none';
    });

    // Load initial saved locations
    fetchSavedLocations();
});
```

```javascript

// Utility Functions
function displayResponseInfo(method, url, status, data) {
    const responseInfo = document.getElementById('response-info');
    responseInfo.textContent = `Method: ${method}
URL: ${url}
Status: ${status}
Timestamp: ${new Date().toLocaleString()}

Data: ${JSON.stringify(data, null, 2)}`;
}
```

```

// GET Request Implementation
async function getWeather() {
    const cityInput = document.getElementById('city-input');
    const city = cityInput.value.trim();

    if (!city) {
        alert('Please enter a city name');
        return;
    }

    const weatherResult = document.getElementById('weather-result');
    weatherResult.innerHTML = 'Loading...';

    try {
        // Constructing the URL with query parameters
        const url = `${WEATHER_API_URL}?q=${encodeURIComponent(city)}&units=metric&appid=${WEATHER_API_KEY}`;

        // Fetch API for GET request
        const response = await fetch(url);
        const data = await response.json();

        // Display response info
        displayResponseInfo('GET', url.replace(WEATHER_API_KEY, 'API_KEY_HIDDEN'), response.status, data);

        if (!response.ok) {
            throw new Error(data.message || 'Failed to fetch weather data');
        }

        // Display weather data
        weatherResult.innerHTML = `
            <div class="weather-card">
                <h3>${data.name}, ${data.sys.country}</h3>
                <div>
                    <strong>Weather:</strong> ${data.weather[0].main} - ${data.weather[0].description}
                </div>
                <div>
                    <strong>Temperature:</strong> ${data.main.temp}°C (Feels like: ${data.main.feels_like}°C)
                </div>
                <div>
                    <strong>Humidity:</strong> ${data.main.humidity}%
                </div>
                <div>
                    <strong>Wind:</strong> ${data.wind.speed} m/s
                </div>
            </div>
            <button id="quick-save" style="background-color: #27ae60;">Save This Location</button>
        `;
```

```
        // Add quick save functionality
        document.getElementById('quick-save').addEventListener('click', () => {
            document.getElementById('location-name').value = `Weather in ${data.name}`;
            document.getElementById('location-city').value = data.name;
            document.getElementById('location-country').value = data.sys.country;
            document.getElementById('location-notes').value = `Temp: ${data.main.temp}°C, Weather: ${data.weather[0].description}`;

            // Switch to the POST tab
            document.querySelector('.tab[data-tab="post"]').click();
        });

    } catch (error) {
        weatherResult.innerHTML = `<div class="weather-card" style="border-left-color: #e74c3c;">
            <h3>Error</h3>
            <p>${error.message}</p>
        </div>`;
    }
}
```

```
122
123 // POST Request Implementation
124 async function saveLocation() {
125     const name = document.getElementById('location-name').value.trim();
126     const city = document.getElementById('location-city').value.trim();
127     const country = document.getElementById('location-country').value.trim();
128     const notes = document.getElementById('location-notes').value.trim();
129
130     if (!name || !city) {
131         alert('Please enter at least a name and city');
132         return;
133     }
134
135     try {
136         // Create the location object
137         const locationData = {
138             title: name,
139             body: JSON.stringify({
140                 city,
141                 country,
142                 notes
143             }),
144             userId: 1 // This is just for JSONPlaceholder API
145         };
146
147         // Fetch API for POST request
148         const response = await fetch(PLACEHOLDER_API_URL, {
149             method: 'POST',
150             headers: {
151                 'Content-Type': 'application/json'
152             },
153             body: JSON.stringify(locationData)
154         });
155
156         const data = await response.json();
157
158         // Display response info
159         displayResponseInfo('POST', PLACEHOLDER_API_URL, response.status, data);
160
161         if (!response.ok) {
162             throw new Error('Failed to save location');
163         }
164
165         // Add to local saved locations (with the ID from the response)
166         const savedLocation = {
167             id: data.id,
168             name,
169             city,
170             country,
171             notes
172         };
```

```
        savedLocations.push(savedLocation);
        renderSavedLocations();

        // Clear form
        document.getElementById('location-name').value = '';
        document.getElementById('location-city').value = '';
        document.getElementById('location-country').value = '';
        document.getElementById('location-notes').value = '';

        // Switch to saved locations tab
        document.querySelector('.tab[data-tab="saved"]').click();

    } catch (error) {
        alert(`Error: ${error.message}`);
    }
}

// Fetch and display saved locations (simulated for JSONPlaceholder)
async function fetchSavedLocations() {
    try {
        // This is a GET request to JSONPlaceholder
        const response = await fetch(`${PLACEHOLDER_API_URL}?userId=1`);
        const data = await response.json();

        // Transform the data to our format - only take first 5 items
        savedLocations = data.slice(0, 5).map(item => {
            // Try to parse the body if it's a valid JSON
            let city = '', country = '', notes = '';

            try {
                const body = JSON.parse(item.body);
                city = body.city || 'Unknown City';
                country = body.country || '';
                notes = body.notes || '';
            } catch (e) {
                // If not valid JSON, use raw body
                city = 'Unknown City';
                notes = item.body;
            }

            return {
                id: item.id,
                name: item.title,
                city,
                country,
                notes
            };
        });

        // Render the locations
        renderSavedLocations();

    } catch (error) {
        console.error('Error fetching saved locations:', error);
    }
}

```

```
// Render saved locations list
function renderSavedLocations() {
    const container = document.getElementById('saved-locations');

    if (savedLocations.length === 0) {
        container.innerHTML = '<p>No saved locations. Add one in the "POST Location" tab.</p>';
        return;
    }

    container.innerHTML = savedLocations.map(location => `
        <div class="location-item" data-id="${location.id}">
            <h3>${location.name}</h3>
            <div><strong>City:</strong> ${location.city}</div>
            ${location.country ? `<div><strong>Country:</strong> ${location.country}</div>` : ''}
            ${location.notes ? `<div><strong>Notes:</strong> ${location.notes}</div>` : ''}
            <div class="location-actions">
                <button class="btn-edit" onclick="editLocation(${location.id})">Edit</button>
                <button class="btn-delete" onclick="deleteLocation(${location.id})">Delete</button>
            </div>
        </div>
    `).join('');
}

```

```
// PUT Request Implementation - Show edit modal
function editLocation(id) {
    const location = savedLocations.find(loc => loc.id === id);

    if (!location) return;

    // Populate the edit form
    document.getElementById('edit-id').value = location.id;
    document.getElementById('edit-name').value = location.name;
    document.getElementById('edit-city').value = location.city;
    document.getElementById('edit-country').value = location.country;
    document.getElementById('edit-notes').value = location.notes;

    // Show the modal
    document.getElementById('edit-modal').style.display = 'block';
}

```

```javascript
// PUT Request Implementation - Update location
async function updateLocation() {
    const id = document.getElementById('edit-id').value;
    const name = document.getElementById('edit-name').value.trim();
    const city = document.getElementById('edit-city').value.trim();
    const country = document.getElementById('edit-country').value.trim();
    const notes = document.getElementById('edit-notes').value.trim();

    if (!name || !city) {
        alert('Please enter at least a name and city');
        return;
    }

    try {
        // Create the updated location object
        const locationData = {
            id,
            title: name,
            body: JSON.stringify({
                city,
                country,
                notes
            }),
            userId: 1
        };

        // Fetch API for PUT request
        const response = await fetch(`${PLACEHOLDER_API_URL}/${id}`, {
            method: 'PUT',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify(locationData)
        });

        const data = await response.json();

        // Display response info
        displayResponseInfo('PUT', `${PLACEHOLDER_API_URL}/${id}`, response.status, data);

        if (!response.ok) {
            throw new Error('Failed to update location');
        }

        // Update the location in our local array
        const index = savedLocations.findIndex(loc => loc.id === parseInt(id));

        if (index !== -1) {
            savedLocations[index] = {
                id: parseInt(id),
                name,
                city,
                country,
                notes
            };

            renderSavedLocations();
        }

        // Hide the modal
        document.getElementById('edit-modal').style.display = 'none';

    } catch (error) {
        alert(`Error: ${error.message}`);
    }
}
```

```javascript

// DELETE Request Implementation
async function deleteLocation(id) {
    if (!confirm('Are you sure you want to delete this location?')) {
        return;
    }

    try {
        // Fetch API for DELETE request
        const response = await fetch(`${PLACEHOLDER_API_URL}/${id}`, {
            method: 'DELETE'
        });

        // Display response info
        displayResponseInfo('DELETE', `${PLACEHOLDER_API_URL}/${id}`, response.status, {
            message: 'Resource deleted successfully'
        });

        if (!response.ok) {
            throw new Error('Failed to delete location');
        }

        // Remove from local array
        savedLocations = savedLocations.filter(loc => loc.id !== id);
        renderSavedLocations();

    } catch (error) {
        alert(`Error: ${error.message}`);
    }
}
```

Step 3: Testing and Usage

To use the application:

1. API Key Setup:
    - Replace `YOUR_OPENWEATHERMAP_API_KEY` in the JavaScript file with your actual API key from OpenWeatherMap
    - Sign up at openweathermap.org if you don't have an API key

2. Running the Application:
    - Open `index.html` in a web browser
    - No server setup is needed for this simple project

3. Testing Each API Operation:
    - **GET**: Enter a city name and click "Get Weather"
    - **POST**: Fill out the location form and click "Save Location"
    - **PUT:** Click "Edit" on a saved location, make changes, and click "Update"
    - **DELETE**: Click "Delete" on a saved location